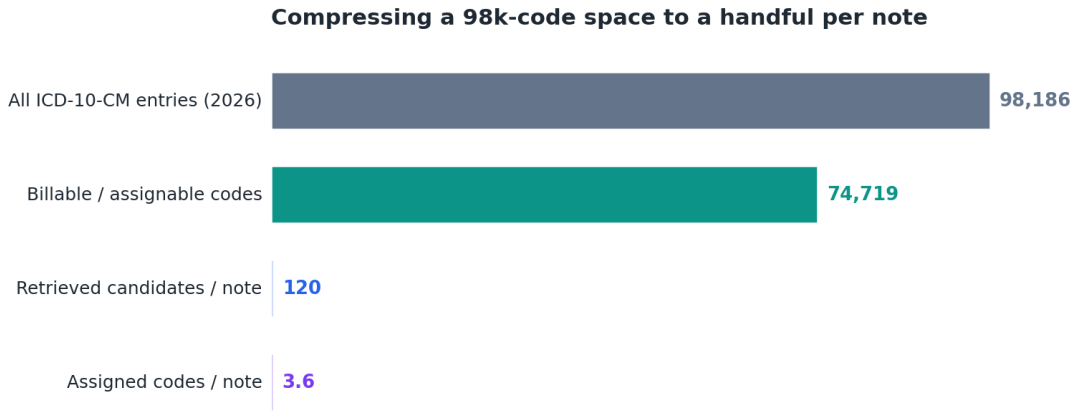
SYSTEM DESIGN & EVALUATION

MediCore

A retrieval-augmented system for automated ICD-10-CM medical coding, powered by a local Ollama model.

19%	75%	23s	3533
RECALL (micro)	RETRIEVAL CEILING	AVG TIME / NOTE	AVG TOKENS / NOTE

Baseline: **qwen2.5:3b** + nomic-embed-text · evaluated on all 54 provided cases.



The core challenge in one picture: the pipeline narrows ~75,000 assignable codes to ~120 candidates per note before the model makes any decision.

SECTION 1

Problem & Framing

Medical coding translates a clinical narrative — a discharge summary, encounter note, or operative report — into standardized codes used for billing, reporting, and analytics. The target vocabulary here is **ICD-10-CM**. The 2026 release contains **98,186** ordered entries, of which **74,719** are valid, billable (assignable) codes; the rest are non-billable header categories in the hierarchy.

Three properties make this hard and shape every design decision:

- **Enormous label space.** ~75k assignable codes cannot fit in any context window, so the system cannot simply "ask the model to pick codes". The label set must be narrowed before generation.
- **Multi-label output.** Each note maps to a variable-size set of codes (in this data: 1–10, mean ≈ 2.9). The system must decide both *which* codes and *how many*.
- **Specificity rules.** ICD-10-CM encodes laterality, encounter type, acuity, and combination concepts inside the code itself (e.g. `S52.502A`). The correct code is often one of dozens of near-identical siblings.

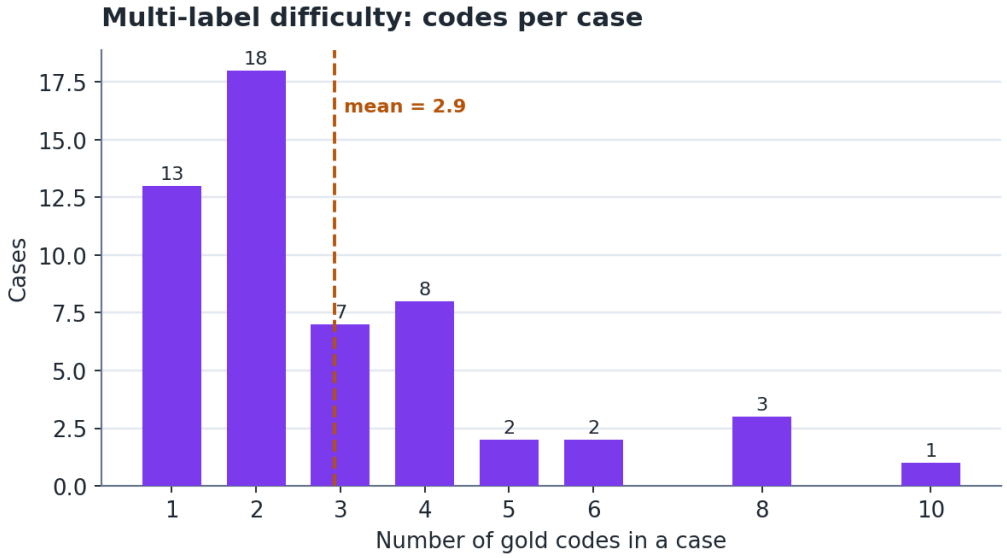


Figure 1 — Distribution of gold codes per case in the provided dataset. Most cases are genuinely multi-label, which is why the system must decide set membership, not a single best label.

The dataset comprises **54 labelled cases** (`icd10_cm_cases.json`) and the official CMS order file (`icd10cm_order_2026.txt`). All 136 unique gold codes are valid billable codes present in the order file, confirming the assignable set is the correct candidate space.

SECTION 2

System Architecture

Because the label space is far too large for direct generation, MediCore uses a **retrieve-then-assign** (retrieval-augmented generation) pipeline. This mirrors how a human coder actually works: read the chart, abstract the codeable concepts, look them up in the index, then select the most specific supported code. The pipeline has an offline index-build stage and a four-step per-note stage.

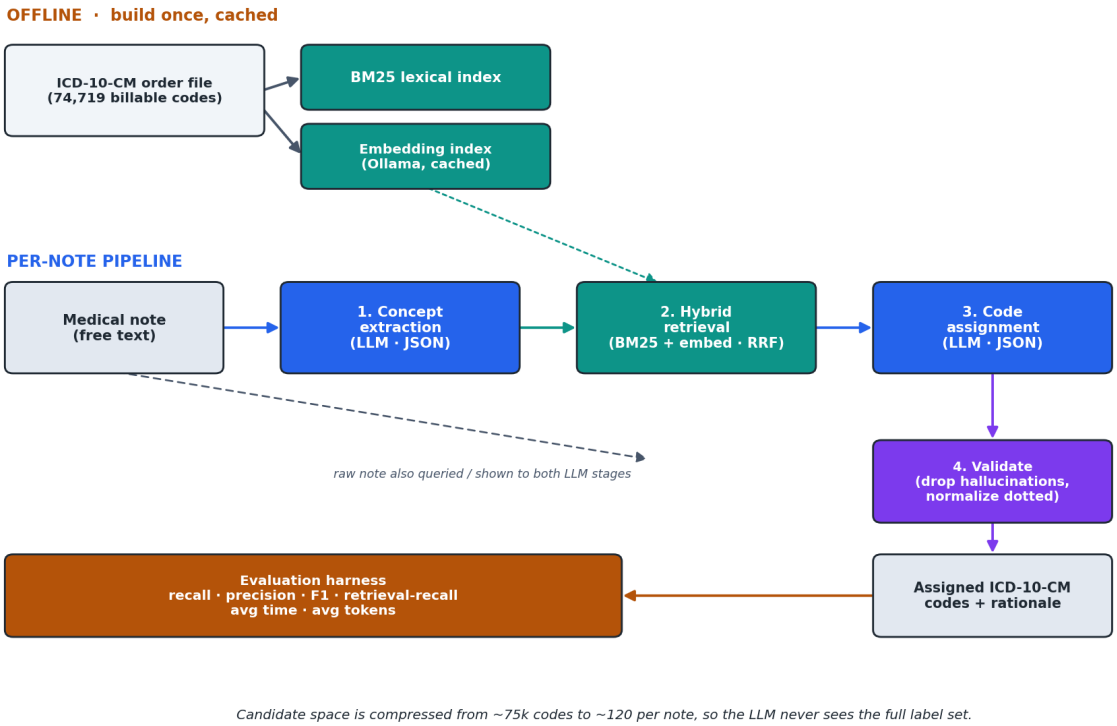


Figure 2 — End-to-end architecture. Retrieval handles recall over the full code space; the LLM is asked only to exercise judgement over a small candidate set.

2.1 · Offline — code knowledge base

The CMS order file is parsed into the 74,719 billable codes, each with its long description. Two indexes are built over the descriptions:

- **BM25 lexical index** — a sparse ranker: fast, model-free, and excellent at matching exact clinical terminology (drugs, organisms, anatomical sites) that appears verbatim in descriptions.
- **Dense embedding index** — each description is embedded with an Ollama model (`nomic-embed-text`) and cached to disk, capturing paraphrase and synonymy BM25 misses ("heart attack" ↔ "myocardial infarction").

2.2 · Per-note pipeline

- **Step 1 — Concept extraction (LLM).** The model reads the note and returns a JSON list of discrete codeable concepts (diagnoses, injuries with laterality, symptoms, procedures, external causes, status/history), turning a long noisy narrative into precise search phrases.
- **Step 2 — Hybrid retrieval.** The whole note and every extracted concept are used as queries. BM25 and embedding results are fused with **Reciprocal Rank Fusion** (RRF) — a rank-based combiner needing no score calibration — into a de-duplicated pool of ~120 candidates.

- **Step 3 — Code assignment (LLM).** The note plus the candidate list (code + description) is given to the model, which selects the supported codes with a short justification each, as JSON, choosing only from the provided candidates.
- **Step 4 — Validation.** Outputs are normalized to dotted form; any code not in the candidate set or not a valid billable code is dropped, eliminating hallucinations by construction.

Why extract concepts with an LLM rather than retrieve on the raw note: on this data, BM25 over the raw note frequently fails to surface the correct code — incidental words like "hospital" and "admitted" pull in place-of-occurrence and external-cause codes — whereas a single extracted phrase such as "severe opioid dependence with withdrawal" retrieves the exact code (`F11.23`) at rank 1.

SECTION 3

Rationale & Alternatives Considered

Alternatives weighed against the chosen RAG design:

- **Direct generation (no retrieval).** Ask the LLM to emit codes from memory. Rejected: small local models don't reliably memorize a 75k-code vocabulary and hallucinate plausible-but-invalid codes, with no validity guarantee.
- **Flat multi-label classifier.** Train a 75k-way classifier. Rejected here: needs a large labelled corpus (54 cases is far too few), and the task constrains us to an Ollama model, not a bespoke trained head.
- **Pure lexical lookup.** BM25 alone. Rejected as a full solution: it retrieves well but cannot judge documentation support, specificity, or de-duplicate sibling codes — that reasoning is what the assignment LLM provides.
- **Hierarchical routing** (chapter → category → code). Viable and complementary, but adds latency and compounds errors across hops; RRF hybrid retrieval reaches the correct leaf directly.

The retrieve-then-assign design makes the LLM responsible only for *judgement over a small candidate set* — what LLMs are good at — while retrieval handles *recall over the huge label space* — what indexes are good at. It is model-agnostic (any Ollama chat model plugs in), needs no training data, and guarantees output validity.

SECTION 4

Trade-offs

- **Retrieval recall is the hard ceiling.** If a gold code is never retrieved, assignment cannot recover it. We therefore bias retrieval toward recall (high K, hybrid, multi-query) at the cost of a larger, noisier candidate list. Section 6 measures this ceiling explicitly.
- **Two LLM calls vs one.** Concept extraction markedly improves retrieval but adds a call (latency + tokens). It is toggleable (`extract_concepts: false`).
- **Candidate-list size.** More candidates raise the recall ceiling but lengthen the assignment prompt and add distractors. 120 is a tunable middle setting.
- **Embeddings vs BM25-only.** Embeddings improve paraphrase recall but need a one-time index build (~75k vectors) and extra memory; a BM25-only mode is provided for constrained environments.
- **Model size vs speed.** A small model keeps latency and memory low but reasons less reliably about specificity; the design swaps models with a one-line config change (quantified in Section 6).

SECTION 5

Assumptions

- The assignable target space is the set of billable (flag = 1) codes in the CMS order file; non-billable headers are never assigned.
- Gold labels are a complete, order-independent set per note; evaluation is set-based (no code sequencing / principal-diagnosis ordering scored).
- Codes are compared in canonical dotted, upper-case form; the order file's undotted codes are normalized to match the gold format.
- Notes are self-contained; no external EHR context, problem list, or prior encounters are available.
- Only an Ollama model is used for all learned components (extraction, assignment, embeddings), per the task constraint.

SECTION 6

Measured Results

Evaluated on all **54** provided cases. Recall is the primary requested metric; precision/F1 give context. *Micro* pools code-level hits across notes (weights by number of codes); *macro* averages per-note scores (weights notes equally).

Metric	Micro	Macro
Recall (primary)	0.190	0.256
Precision	0.155	0.192
F1	0.170	0.210

Operational metric	Value
Cases evaluated	54
Retrieval recall (candidate ceiling)	0.754
Average time per note	23.09 s
Average total tokens per note	3533
- prompt tokens	3275
- completion tokens	259

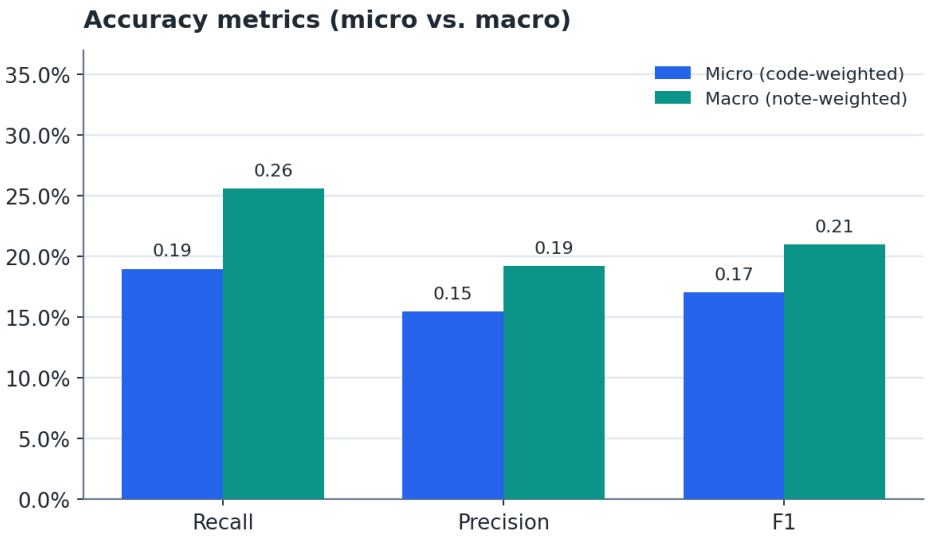


Figure 3 — Accuracy metrics, micro vs. macro.

6.1 · Where recall is lost

The single most useful diagnostic is decomposing recall into a retrieval stage and an assignment stage. Retrieval surfaces the majority of gold codes into the candidate pool (the *ceiling*); the remaining loss is the assignment model failing to select a retrieved gold code — typically choosing a near-identical sibling that differs only in laterality or specificity.

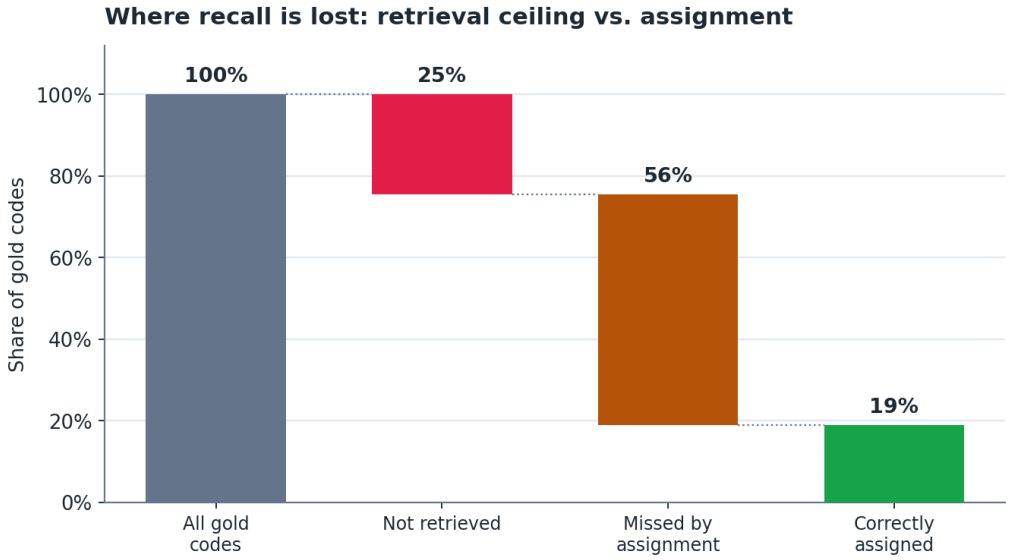


Figure 4 — Recall decomposition. The gap from 100% to the ceiling is what retrieval never surfaced; the gap from the ceiling to the green bar is what the assignment model lost. This tells you exactly where to invest next.

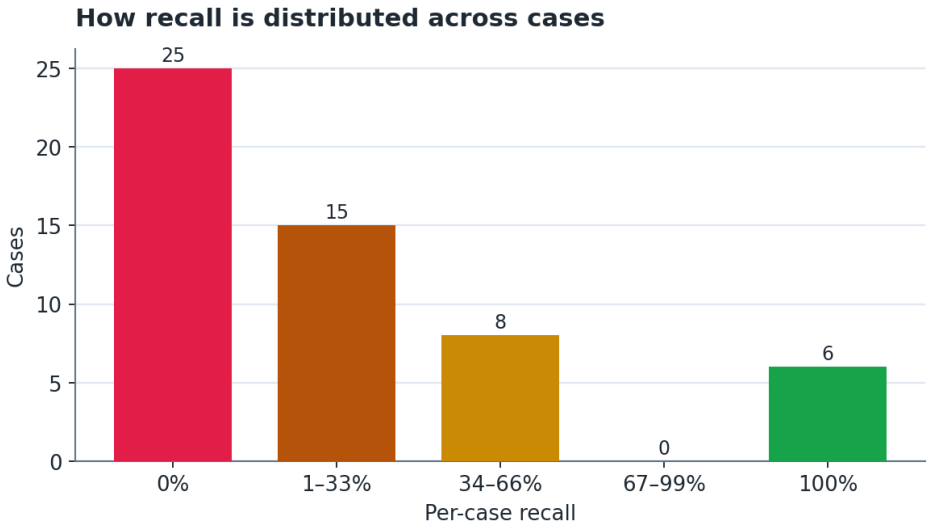


Figure 5 — Per-case recall distribution: the system fully or partly codes many cases while missing others entirely, rather than degrading uniformly.

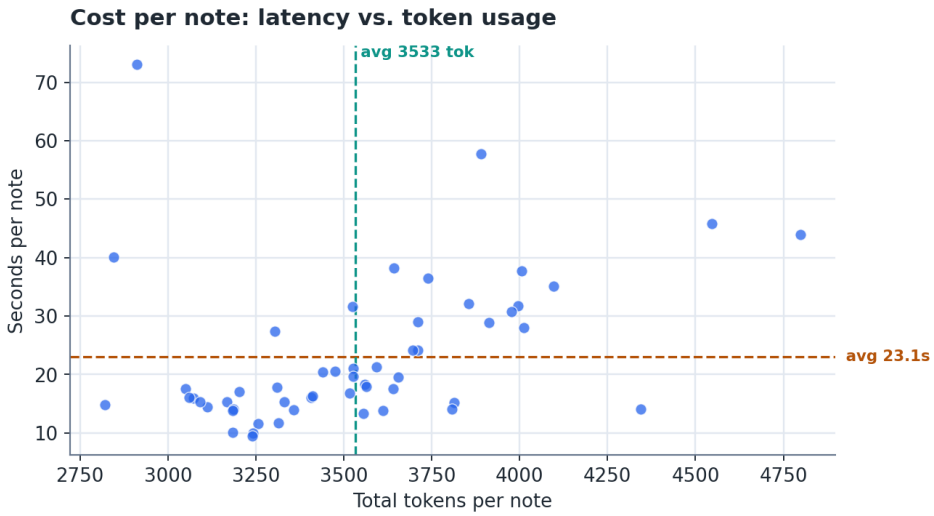


Figure 6 — Per-note cost. Prompt tokens dominate (the candidate list); completion is small. Latency is governed by the two LLM calls on CPU.

Token usage is read directly from Ollama's response fields (`prompt_eval_count`, `eval_count`); timing is wall-clock per note including retrieval. These figures scale with the chosen model and candidate-list size, not with the design.
A model-comparison section (qwen2.5:3b vs. llama3.1:8b) appears here when `reports/eval_llama_summary.json` *is present — run* `python scripts/run_eval.py --model llama3.1:8b --prefix eval_llama`.

SECTION 7

How to Improve the System Further

Ordered roughly by expected return on effort:

- **Stronger / fine-tuned assignment model.** Figures 4 and 7 show the dominant remaining error is specificity judgement, not retrieval — the highest-leverage change is a larger or LoRA-tuned Ollama model.
- **Learned reranker.** A cross-encoder between retrieval and assignment pushes gold codes to the top of the candidate list, raising the achievable ceiling per token.
- **Ontology-aware retrieval.** Exploit the ICD-10 hierarchy and the Alphabetic Index / Includes-Excludes notes to expand queries and constrain candidates to coherent sub-trees (raises the ceiling above its current value).
- **Coding-guideline injection.** Supply the ICD-10-CM Official Guidelines (sequencing, 7th-character, combination rules) as context to improve specificity and reduce invalid combinations.
- **Constrained / grammar decoding.** Force assignment output onto the candidate-code grammar so malformed or out-of-set codes are impossible rather than filtered afterward.
- **Self-consistency & verification.** Sample multiple assignments and vote, or add a verifier pass checking each code against the documentation before emitting it.
- **Human-in-the-loop.** Surface per-code justifications and confidence so a coder can accept/reject — turning the tool assistive and generating new training data.
- **Scale & robustness.** Batch embedding, an ANN index (FAISS) for sub-millisecond retrieval at full scale, and evaluation on a larger, multi-institution corpus (54 cases is indicative, not statistically tight).

Reproducibility. Every number and figure in this document is produced from `reports/eval_summary.json` by `python scripts/run_eval.py`, then `python docs/make_charts.py` and `python docs/generate_writeup.py`. See the README for configuration and setup.